

Tái Hiện DeSFAM và Triển Khai Trên Cụm Kubernetes Trực Tiếp với Cilium Tetragon

Minh

KMA — Báo Cáo Chuyên Đề (2025–2026)

Tóm tắt nội dung—Chúng tôi trình bày một nghiên cứu tái hiện và triển khai độc lập của SyscallAD, bộ phát hiện bất thường syscall ở trung tâm của framework DeSFAM. Trên bộ dữ liệu DongTing, kết quả công bố $F_1 = 0,92$ được tái hiện trong khoảng nhiễu lấy mẫu ($F_1 = 0,932$ với tập hợp VAE+iForest, $F_1 = 0,963$ với VAE đơn lẻ) sau khi thay một bước chuẩn hoá không được nêu trong pipeline tham chiếu bằng một bộ scaler dải lượng phân vị nguyên tắc. Ngoài việc tái hiện, chúng tôi thu một bộ dữ liệu syscall có nhãn với $\approx 16k$ sự kiện trực tiếp từ luồng gRPC của Cilium Tetragon trên ba khối công việc container kiểu sản xuất, triển khai bộ phát hiện đã huấn luyện như một tiến trình tiêu thụ gRPC trong-cụm, và đánh giá sáu biến thể mô hình trên ba tập kiểm tra. Hai giới hạn cấu trúc của pipeline công bố được phơi bày. Thứ nhất, VAE huấn luyện trên DongTing tổng quát hoá kém với lưu lượng syscall container thật (AUC = 0,135, thấp hơn ngẫu nhiên về thống kê), và huấn luyện lại cùng kiến trúc trên dữ liệu Container khôi phục AUC = 1,0. Thứ hai, thành phần iForest của tập hợp đóng góp tín hiệu phân biệt không đáng kể dưới cả hai phân phối huấn luyện, trong khi các baseline có-giám-sát (LSTM, 1-D CNN) trên cùng không gian đặc trưng cạnh tranh được. Độ trễ chấm điểm theo cửa sổ là 9–18 ms trên một lõi CPU đơn, chủ yếu do xây dựng đặc trưng. Chúng tôi đưa ra khuyến nghị cụ thể cho việc hiệu chỉnh ngưỡng trong-môi-trường và trọng số tập hợp tại thời điểm triển khai.

I. GIỚI THIỆU

Việc triển khai container dùng chung một nhân Linux khiến giao diện lời gọi hệ thống (syscall) trở thành bề mặt tấn công có giá trị cao. Framework DeSFAM [1] đề xuất SyscallAD, bộ phát hiện bất thường lai gồm Bộ Mã Hóa Biến Phân (VAE) và Isolation Forest (iForest) trên đặc trưng thủ công của các cửa sổ syscall cố định độ dài, triển khai dòng (inline) qua eBPF/Tetragon [2]. Trên bộ dữ liệu DongTing [3], nhóm tác giả gốc công bố $F_1 = 0,92$, độ chính xác 0,94 và độ thu hồi 0,90 với độ trễ phát hiện dưới mili-giây.

Hai đặc tính của kết quả đó là động lực cho một phép tái hiện độc lập. Thứ nhất, mô tả công bố của bước hợp nhất điểm số bỏ qua bộ chuẩn hoá áp dụng lên hai kênh điểm thành phần — một chi tiết hoá ra quyết định việc tập hợp được hợp nhất hay sụp đổ. Thứ hai, lớp lãnh tính của DongTing được lấy từ các bộ kiểm thử nhân Linux

(LTP, Kselftests, OpenPOSIX, glibc) và lớp tấn công từ các PoC của syzkaller [4]; cả hai phân phối đều không đại diện cho lưu lượng syscall do các khối công việc container thực sinh ra. Do đó việc các chỉ số công bố có tổng quát hoá sang triển khai cloud-native thực tế hay không là một câu hỏi mở.

Báo cáo này trả lời các câu hỏi nghiên cứu sau:

- CH1. Bộ phát hiện SyscallAD có tái hiện được trên DongTing với cấu hình đã công bố hay không?
- CH2. Một mô hình SyscallAD huấn luyện trên DongTing có chuyển giao được tới vết syscall thu từ các khối công việc container kiểu sản xuất hay không?
- CH3. Các bộ phân loại chuỗi có-giám-sát (LSTM, 1-D CNN) so sánh như thế nào với tập hợp VAE + iForest không-giám-sát trên cùng không gian đặc trưng?
- CH4. Pipeline triển khai đạt độ trễ phát hiện nào trước các vết khai thác CVE thật và các hành vi tấn công phổ biến, đo end-to-end qua Tetragon?

Các đóng góp của báo cáo: (i) một bản tái hiện mã nguồn mở của SyscallAD kèm bản sửa lỗi chuẩn hoá đã được ghi chép trong pipeline tham chiếu (Mục IV); (ii) một bộ phát hiện nguyên gRPC tiêu thụ sự kiện Cilium Tetragon ở 9–13 ms cho mỗi cửa sổ 200 syscall trên một lõi CPU đơn (Mục V); (iii) một tập dữ liệu syscall có nhãn thu trực tiếp từ các khối công việc container kiểu sản xuất trong Kubernetes, cùng với hai mô hình huấn luyện lại (Mục III); và (iv) một phép đánh giá so sánh năm biến thể mô hình trên ba tập kiểm tra kèm độ trễ phát hiện theo từng CVE (Mục VI).

II. BỐI CẢNH VÀ CÔNG TRÌNH LIÊN QUAN

A. Phát hiện bất thường dựa trên syscall

Forrest và cộng sự [5] đã giới thiệu chuỗi syscall như một dấu vân hành vi quá trình. Các công trình tiếp theo khai thác mô hình chuỗi (LSTM/CNN) và phương pháp không-giám-sát như autoencoder và isolation forest [6], [7]. Nhóm tác giả DeSFAM kết hợp hai phương pháp sau thành tập hợp cộng, với trực giác rằng tái tạo VAE bao phủ phần lớn hành vi lành tính trong khi iForest bắt các ngoại lệ hiếm.

B. Pipeline SyscallAD

SyscallAD vận hành trên đặc trưng 174 chiều tính trên các cửa sổ syscall cố định (§III). VAE không-giám-sát

được huấn luyện trên chuỗi lệnh tính và chấm điểm cửa sổ kiểm tra bằng lỗi tái tạo; iForest huấn luyện trên cùng tập lệnh tính và chấm điểm bằng độ sâu cô lập. Hai điểm thô được chuẩn hoá và hợp nhất: $A(W) = \alpha A_{VAE}(W) + (1 - \alpha) A_{iForest}(W)$ với $\alpha = 0,7$.

C. eBPF và Cilium Tetragon

Tetragon gắn kprobe eBPF vào điểm vào syscall và phối một API gRPC dòng (FineGuidanceSensors.GetEvents). Mỗi sự kiện mang định danh tiến trình và pod, mã syscall, cùng đối số tùy chọn. Tiêu thụ API trực tiếp được ưu tiên hơn so với phân tích JSON xuất ra vì lược đồ gRPC có kiểu dữ liệu rõ ràng và thứ tự sự kiện được bảo đảm.

III. PHƯƠNG PHÁP LUẬN

A. Trích đặc trưng

Mỗi quyết định chấm điểm tính trên cửa sổ $W = 200$ định danh syscall với bước trượt $S = 50$. Cửa sổ được ánh xạ về véc-tơ 174 chiều theo năm nhóm của thiết kế DeSFAM:

- 60 tần suất syscall (số đếm tương đối của các định danh syscall phổ biến nhất trong DongTing);
- 40 cờ nhị phân phân biệt (sự hiện diện của các định danh thiên về tấn công);
- 8 thống kê phân phối gồm độ dài cửa sổ, độ đa dạng định danh duy nhất, và entropy Shannon của phân phối định danh trong cửa sổ;
- 40 tần suất chuyển tiếp bigram (tần suất cặp liên kề phân biệt nhất);
- one-hot phiên bản nhân 26 chiều.

StandardScaler khớp trên cửa sổ huấn luyện DongTing được áp dụng nhất quán trong mọi thí nghiệm để bảo đảm so sánh tương đương.

B. Các biến thể mô hình

Sáu biến thể mô hình được đánh giá. Siêu tham số được liệt kê ở Bảng I. Hậu tố dữ liệu huấn luyện là một phần của tên biến thể để phép so sánh không nhập nhằng.

Các biến thể không-giám-sát (VAE, iForest, VAE+iForest) chỉ cần chuỗi lệnh tính nên cho phép lựa chọn sạch giữa hai phân phối huấn luyện, DongTing hoặc Container. Các biến thể có-giám-sát (LSTM, 1-D CNN) cần mẫu có nhãn của cả hai lớp; riêng tập tấn công Container chỉ chứa ≈ 40 cửa sổ tấn công, không đủ. Do đó các mô hình có-giám-sát được huấn luyện trên hợp DongTing+Container.

C. Bộ dữ liệu

a) *DongTing*.: Bộ dữ liệu đã công bố [3] có 18.966 chuỗi (6.850 lệnh tính, 12.116 độc hại) lấy từ 26 phiên bản nhân Linux. Chúng tôi dùng phân chia DTDS công bố (14.843 huấn luyện, 1.812 xác nhận, 2.311 kiểm tra) và 5.487 chuỗi lệnh tính của tập huấn luyện cho các phép khớp không-giám-sát.

b) *Container*.: Tập dữ liệu ≈ 16.200 syscall được thu trực tiếp từ luồng gRPC của Cilium Tetragon trên cụm Kubernetes thí nghiệm (Mục V). Thành phần lệnh tính được bắt từ ba khối công việc dưới tải thực tế: **nginx+wrk**, **redis+redis-benchmark** và **postgres+pgbench**. Thành phần tấn công được bắt từ năm khối công việc: vết khai thác CVE-2021-4034 (Polkit pkexec) 60 vòng, vết khai thác CVE-2022-0847 (Dirty Pipe) 60 vòng, một driver reverse shell, một driver tar exfil, và một mô phỏng miner. Sau khi trượt cửa sổ ($W = 200$, $S = 50$) và áp dụng scaler khớp trên DongTing, tập Container gồm 296 cửa sổ với tỉ lệ tấn công 28,0%, phân chia 70/15/15 stratified theo khối công việc (Bảng II).

D. Quy trình đánh giá

Tất cả biến thể được đánh giá trên cùng hai tập kiểm tra — kiểm tra DongTing (2.311 cửa sổ, 70,7% tấn công) và kiểm tra Container (45 cửa sổ, 28,9% tấn công). Với mỗi biến thể, ngưỡng quyết định được chọn theo cực đại Youden- J trên tập xác nhận của phân phối huấn luyện gốc. Các chỉ số báo cáo gồm AUC, Average Precision (AP), F_1 , độ chính xác và độ thu hồi.

Với câu hỏi nghiên cứu CH4, bốn vết độc hại (hai khai thác CVE, exfil, reverse shell) được chấm điểm theo từng cửa sổ. *Độ trễ phát hiện* của một mô hình trên một vết là số cửa sổ từ đầu vết đến cửa sổ đầu tiên có điểm vượt ngưỡng của mô hình; mô hình không vượt được báo cáo là không phát hiện.

Bảng I
SIÊU THAM SỐ CỦA SÁU BIẾN THỂ. TẤT CẢ VẬN HÀNH TRÊN CÙNG CỬA SỐ 174 CHIỀU (§III-A) VỚI $W = 200$, $S = 50$ VÀ STANDARDSCALER KHỚP TRÊN TẬP HUẤN LUYỆN DONGTING.

Biến thể	Kiến trúc / Thuật toán	Cấu hình huấn luyện
VAE (DongTing)	Enc/Dec 174→128→64→24→64→128→174, SELU, dropout 0,2	Chỉ chuỗi lành tính DongTing (5.487); Adam 10^{-3} ; batch 256; ≤ 80 epoch; dừng sớm patience 10; 3 seed; MSE+KL; RobustScaler p_1-p_{99} trên điểm xác nhận
VAE (Container)	Cùng kiến trúc encoder/decoder	Chỉ tập con lành tính Container (149 cửa sổ); cùng vòng huấn luyện; ≤ 50 epoch; dừng sớm patience 8; 3 seed
iForest (DongTing)	Scikit-learn IsolationForest	300 cây; <code>contamination='auto'</code> ; <code>random_state=42</code> ; chỉ chuỗi lành tính DongTing; RobustScaler p_1-p_{99}
VAE+iForest (DongTing)	$A = \alpha A_{VAE} + (1-\alpha)A_{iForest}$, cả hai huấn luyện trên DongTing	$\alpha = 0,7$ [1]; cả hai điểm chuẩn hoá bằng RobustScaler trước khi hợp nhất
LSTM (DongTing+Container)	Reshape(174,1) → LSTM(64) → Dense(32, ReLU) → Drop 0,3 → Dense(1, sigmoid)	DongTing+Container có nhãn (cả 2 lớp); BCE; Adam 10^{-3} ; batch 256; ≤ 50 epoch; dừng sớm patience 6 theo val-AUC
1-D (DongTing+Container)	CNN Reshape(174,1) → Conv1D(32, k=5) → MaxPool1D(2) → Conv1D(64, k=3) → GlobalMaxPool → Dense(32, ReLU) → Drop 0,3 → Dense(1, sigmoid)	Cùng cấu trúc có-giám-sát với LSTM (DongTing+Container)

Bảng II
PHÂN CHIA TẬP DỮ LIỆU CONTAINER.

Phân chia	Cửa sổ	Tỉ lệ tấn công
huấn luyện	207	28,0%
xác nhận	44	27,3%
kiểm tra	45	28,9%

Bảng III
TÁI HIỆN TRÊN TẬP KIỂM TRA DONGTING (2.311 CỬA SỐ, 70,7% TẤN CÔNG).

Mô hình	AUC	AP	F_1	Độ chính xác
iForest (DongTing)	0,882	0,949	0,915	0,848
VAE (DongTing)	0,960	0,970	0,963	0,937
VAE+iForest (DongTing, $\alpha = 0,7$)	0,921	0,960	0,932	0,876
Bài báo [1]	0,94	0,87	0,92	0,94

IV. TÁI HIỆN SYSCALLAD TRÊN DONGTING

A. Lỗi chuẩn hoá trong pipeline tham chiếu

Bản hiện thực tham chiếu phát hành cùng DongTing chuẩn hoá hai kênh điểm thô bằng thống kê min-max khớp trên điểm tập huấn luyện. Trên phân phối DongTing, lỗi tái tạo VAE thô trải qua xấp xỉ sáu cấp độ độ lớn (giá trị hợp lệ ≈ 4 , ngoại lệ $\approx 1,5 \times 10^7$), nên min-max nén phần lớn phân phối vào một dải mỏng quanh giá trị 0. Ngay cả với $\alpha = 0,7$, kết quả tập hợp *suy biến* thành kết quả iForest: cả hai đều cho $F_1 = 0,9151$ với ma trận nhầm lẫn trùng khớp trên tập kiểm tra DongTing.

Chúng tôi thay min-max bằng dải lượng phân vị khớp trên cùng điểm huấn luyện:

$$\hat{x} = \text{clip}\left(\frac{x - P_1(x_{\text{ref}})}{P_{99}(x_{\text{ref}}) - P_1(x_{\text{ref}}) + \epsilon}, 0, 1\right). \quad (1)$$

Trên cùng dữ liệu, biên trên của kênh VAE giảm từ $1,5 \times 10^7$ xuống $2,18 \times 10^3$, đóng góp của VAE được bảo toàn trong hợp nhất, và F_1 tập hợp tăng lên 0,9318. Bài học có giá trị tổng quát: khi một tập hợp cộng kết hợp điểm từ các bộ phát hiện không đồng nhất, dải động của từng kênh phải được khảo sát trước khi chọn bộ chuẩn hoá.

B. Kết quả tái hiện

Bảng III báo cáo các chỉ số trên tập kiểm tra DongTing. F_1 tập hợp (0,932) nằm trong khoảng nhiễu lấy mẫu so với 0,92 đã công bố. Thành phần VAE đơn lẻ đạt $F_1 = 0,963$, vượt tập hợp ở mọi chỉ số tổng hợp trừ độ thu hồi. Đây là chứng cứ đầu tiên cho thấy công thức tập hợp công bố không thực sự thống trị thành phần đơn lẻ mạnh nhất của nó.

Kết quả này trả lời **CH1** ở dạng khẳng định, có điều kiện là thay thế bộ chuẩn hoá min-max bị hỏng bằng Phương trình 1.

V. TRIỂN KHAI QUA CILIUM TETRAGON

A. Kiến trúc

Bộ phát hiện đã huấn luyện được đóng gói thành một image Python duy nhất, không phơi công và tiêu thụ sự kiện qua dịch vụ gRPC của Tetragon trong cụm. Với mỗi sự kiện, bộ lọc không gian tên được áp dụng, định danh syscall được trích từ oneof `ProcessKprobe`, `ProcessTracepoint`, `ProcessExec` hoặc `ProcessExit`, và thêm vào hàng đợi cuộn theo từng pod. Mỗi $S = 50$ syscall mới, véc-tơ đặc trưng 174 chiều đầy đủ được tính lại và chấm điểm. Phán định được phát ra dưới dạng dòng stdout có cấu trúc, được hệ thống log của cụm ingest.

B. Ghi nhận triển khai

Ba mục cấu hình yêu cầu can thiệp đặc thù nền tảng trên Docker Desktop Kubernetes (nhân 6,12, arm64):

- `/sys/fs/bpf` của máy chủ được mount riêng trên máy ảo LinuxKit, làm cho DaemonSet của Tetragon không gắn được dù dùng mount propagation Bidirectional hay HostToContainer. Tình trạng này được giải quyết bởi một pod đặc-quyền một-lần dùng `nsenter` vào không gian mount của máy chủ và remount BPF FS dạng shared.
- Mục tiêu kprobe trong `TracingPolicy` phải dùng dạng không tiền tố (`sys_execve`) với `syscall: true`.

Bảng IV
PHÁN ĐỊNH V1 TRÊN SÁU KHỐI CÔNG VIỆC ĐA DẠNG VỚI
VAE+iFOREST (DongTing), $\alpha = 0,7$, NGUỖNG 0,077.

Khối công việc	n	TB VAE	TB iForest	TB kết hợp
anom-execbomb	69	0,524	0,523	0,524
anom-netscan	7	0,408	0,594	0,464
anom-kernel	24	0,274	0,485	0,337
anom-privesc	228	0,222	0,536	0,316
anom-benign	47	0,170	0,514	0,273
anom-ptrace	241	0,076	0,420	0,179

Tetragon cung cấp tiền tố đặc thù kiến trúc khi nạp. Mã hoá cứng `__x64_sys_*` khiến chính sách nạp im lặn dưới dạng no-op trên arm64.

- Helm chart binding máy chủ gRPC vào unix socket mặc định. Đặt `tetragon.grpc.address=0.0.0.0:54321` và phối Service ClusterIP trên cổng đó là bắt buộc để API tiếp cận được từ pod khác.

C. Đặc tính hiệu năng

Độ trễ chấm điểm theo cửa sổ, đo end-to-end từ khi sự kiện đến qua xây dựng đặc trưng và suy luận mô hình, nằm trong 9–18 ms cho mọi biến thể trên một lõi CPU đơn. Năm lần lấy mẫu được dùng cho mỗi dự đoán VAE để ổn định ước lượng lỗi tái tạo. Các biến thể có-giám-sát chậm hơn không đáng kể (24–34 ms) do bước convolution thứ hai.

VI. ĐÁNH GIÁ THỰC NGHIỆM

A. Kiểm thử đa dạng sáu khối công việc (v1)

Bộ phát hiện v1 đã triển khai với tập hợp VAE+iForest (DongTing) gốc được chạy với sáu khối công việc trong không gian tên `demo`: một bomb fork-exec, một bộ quét mạng, vòng lặp I/O `dd/cat`, vòng lặp leo thang đặc quyền `unshare+mount`, vòng lặp gắn trình gỡ lỗi, và đối chứng lành tính `curl`. Bảng IV tổng hợp điểm theo cửa sổ quan sát được.

Hai mẫu hiện rõ trong Bảng IV trả lời **CH2** ở dạng phủ định. Thứ nhất, đối chứng lành tính được chấm điểm trên ngưỡng công bố bởi mọi thành phần, cho thấy ngưỡng tinh chỉnh theo DongTing không chuyển sang đường nền container thật. Thứ hai, khối gắn trình gỡ lỗi (sử dụng `ptrace` dày đặc) được chấm điểm *thấp hơn* đối chứng lành tính, không phải cao hơn. Chúng tôi diễn giải đây là hệ quả của việc VAE không-giám-sát đã học, từ lớp lành tính bộ kiểm thử nhân của DongTing, rằng lưu lượng `ptrace` dày đặc là bình thường — vì lớp đó thực tế bảo hoà luồng làm việc gỡ lỗi.

B. Phân tích so sánh mô hình

Bảng V báo cáo các chỉ số của sáu biến thể trên cả tập kiểm tra DongTing và tập kiểm tra Container.

VAE (DongTing) sụp đổ trên tập kiểm tra Container với $AUC = 0,135$, thấp hơn ngẫu nhiên về mặt thống kê. Huấn luyện lại cùng kiến trúc trên tập con lành tính Container

Bảng V
HIỆU NĂNG THEO TỪNG TẬP KIỂM TRA. DT = KIỂM TRA DONGTING (2.311 CỬA SỐ, 70,7% TẦN CÔNG); CONT. = KIỂM TRA CONTAINER (45 CỬA SỐ, 28,9% TẦN CÔNG).

Mô hình	AUC DT	AUC Cont.	F_1 DT	F_1 Co
VAE (DongTing)	0,960	0,135	0,952	0,
VAE (Container)	0,687	1,000	0,842	1,
iForest (DongTing)	0,882	1,000	0,832	1,
LSTM (DongTing+Container)	0,998	1,000	0,992	1,
1-D CNN (DongTing+Container)	1,000	1,000	1,000	1,

Bảng VI
ĐỘ TRỄ PHÁT HIỆN THEO CỬA SỐ (‘-’ = KHÔNG PHÁT HIỆN TRONG VẾT). DT = HUẤN LUYỆN DONGTING, CONT. = HUẤN LUYỆN CONTAINER, $\dagger$ = MÔ HÌNH CÓ-GIÁM-SÁT HUẤN LUYỆN TRÊN DONGTING+CONTAINER.

Vết	n_w	VAE (DT)	VAE (Cont.)	iForest (DT)	LSTM $\dagger$
CVE-2021-4034	63	0	—	0	4
CVE-2022-0847	8	0	0	0	—
exfil	7	0	0	0	—
revshell	5	0	4	0	—

đưa AUC Container về 1,000. Các baseline có-giám-sát đạt chỉ số tổng hợp mạnh nhất, nhưng kèm lưu ý nêu ở Mục III-B: khoảng 70% số cửa sổ tần công Container đã được thấy trong huấn luyện, làm tăng các chỉ số Container so với một đánh giá ngoài-mẫu thực sự.

Bảng chứng kết hợp từ Bảng III và Bảng V trả lời **CH3**: trên không gian đặc trưng có sẵn dữ liệu tần công có nhân, các bộ phân loại có-giám-sát đơn giản (LSTM, 1-D CNN) ngang hoặc vượt tập hợp không-giám-sát đã công bố. Thành phần iForest đóng góp tối thiểu; điểm của cột iForest trong Bảng IV gần như không đổi theo khối công việc, cho thấy tập hợp thừa kế phần lớn sức mạnh phân biệt từ VAE.

C. Độ trễ phát hiện CVE

Bảng VI báo cáo độ trễ phát hiện (theo số cửa sổ) cho bốn vết độc hại của Mục III-C.

Các biến thể huấn luyện DongTing và iForest đánh dấu mọi vết độc hại ngay cửa sổ 0, nhưng để diễn giải cần đối chiếu với đường nền đa dạng ở Mục VI-A: các biến thể này cũng đánh dấu đối chứng lành tính. Độ trễ bằng 0 chỉ là phụ phẩm của việc gán nhãn dương không phân biệt chứ không phải phát hiện phân biệt. LSTM có-giám-sát bỏ sót ba trên bốn vết. 1-D CNN có-giám-sát phát hiện cả bốn ngay cửa sổ 0 với điểm cực đại chuẩn hoá nằm trong $[0,95, 0,99]$. Đây là câu trả lời vận hành cho **CH4**: khi mô hình có-giám-sát đã thấy các ví dụ tần công tương đương trong huấn luyện, cửa sổ chấm điểm đầu tiên là đủ.

D. Triển khai lại sản xuất

Để xác nhận sự cải thiện về chất trong triển khai, image bộ phát hiện được rebuild với cờ `--model-variant` và triển khai lại với VAE (Container) cùng ngưỡng quyết định 0,5 chọn từ phân phối điểm v2. Bài kiểm thử cùng sáu khối công việc được lặp lại.

Bảng VII
KIỂM THỬ TRIỂN KHAI SÁU KHỐI CÔNG VIỆC, TRƯỚC VÀ SAU. TRƯỚC:
VAE+iFOREST (DONGTING) NGƯỠNG 0,077. SAU:
VAE (CONTAINER) NGƯỠNG 0,5.

Khối công việc	TB trước	TB sau	ATK % sau
anom-netscan	0,464	1,000	100%
anom-kernel	0,337	1,000	100%
anom-execbomb	0,524	1,000	100%
anom-privesc	0,316	0,867	85%
anom-pttrace	0,179	0,303	0%
anom-benign	0,273	0,218	0%

Bốn dạng lỗi đã nhận diện trước đó được giải quyết. Đối chứng lành tính không còn trên ngưỡng; điểm của khối **pttrace** nay vượt điểm đối chứng lành tính; biên độ giữa tấn công xấu nhất và lành tính tốt nhất rơi từ $\approx 3 \times$ ở v1 lên $\approx 4,6 \times$ ở v2; và bốn bất thường thật được gắn nhãn ATTACK với tỉ lệ $\geq 85\%$ trong khi cả hai khối lớp lành tính vẫn yên tĩnh.

VII. THẢO LUẬN

Kết quả chính là việc lựa chọn dữ liệu huấn luyện chi phối lựa chọn kiến trúc trong lớp bài toán này. Giữ nguyên không gian đặc trưng 174 chiều, kiến trúc VAE và bộ chuẩn hoá RobustScaler, huấn luyện lại chỉ trên dữ liệu Container đưa AUC Container từ dưới ngẫu nhiên lên 1,0. Lớp lành tính của DongTing không đại diện cho mẫu syscall container sản xuất vì nó được tạo bởi các bộ kiểm thử nhân vận hành các syscall hiếm dùng — bao gồm các luồng **pttrace** dày đặc làm ô nhiễm phân phối lành tính đã học. Người triển khai SyscallAD trên khối công việc thật do đó nên dự liệu một pha huấn luyện trong-môi-trường thay vì dựa vào trọng số huấn luyện trên DongTing.

Một kết quả phụ liên quan đến cấu thành tập hợp. Cả ở phép tái hiện trên kiểm tra DongTing (Bảng III) và phép kiểm thử đa dạng ngoài phân phối (Bảng IV), thành phần VAE đơn lẻ vượt tập hợp $\alpha = 0,7$ VAE+iForest ở các chỉ số tổng hợp. Điểm của iForest theo cửa sổ nằm trong dải hẹp [0,42, 0,59], đóng góp ít tín hiệu phân biệt. Chúng tôi giả thiết rằng lựa chọn $\alpha = 0,7$ của bài báo chỉ đúng dưới một bộ chuẩn hoá không được nêu rõ, và dưới bộ chuẩn hoá robust nguyên tắc của Phương trình 1, chế độ $\alpha \rightarrow 1$ được ưu tiên.

a) *Các nguy cơ với tính hợp lệ.*: Thứ nhất, tập dữ liệu Container nhỏ (296 cửa sổ, $\approx 16k$ syscall). Các baseline có-giám-sát hưởng lợi không cân xứng vì quan sát hầu hết phân phối tấn công trong huấn luyện; các kết quả kiểm tra Container của chúng do đó nên đọc như giới hạn trên. Một kho tấn công Container lớn hơn sẽ thắt chặt phép so sánh. Thứ hai, TracingPolicy chỉ hook một tập tuyến chọn 20 syscall, làm nén tỉ lệ sự kiện thực tế theo từng pod xuống ≈ 10 sự kiện/giây và thiên lệch phân phối được bắt về phía các nguyên hàm điều khiển luồng và tạo tiến trình. Một chính sách raw-tracepoint sẽ cho vết dày hơn với khối lượng sự kiện cao hơn. Thứ ba, mọi phép đo được thực hiện trên một node Docker Desktop đơn (LinuxKit, arm64); các con số độ trễ tuyệt đối không nhất thiết chuyển sang cụm nhiều node.

VIII. GIỚI HẠN VÀ CÔNG VIỆC TƯƠNG LAI

Tập dữ liệu Container đủ để bác bỏ mô hình huấn luyện trên DongTing trong triển khai nhưng không đủ để huấn luyện một baseline có-giám-sát mạnh chỉ trên Container. Bước tự nhiên tiếp theo là mở rộng pipeline thu thập dữ liệu sang nhiều node và nhiều ngày lưu lượng. Với cơ sở đó, bốn thí nghiệm tiếp theo gợi ý sẵn sàng: (i) thay VAE+iForest bằng chỉ-VAE ở ngưỡng triển khai-thời để kiểm chứng liệu giá trị biên của tập hợp có biện minh được chi phí; (ii) quét $\alpha \in \{0,7; 0,8; 0,9; 1,0\}$ dưới Phương trình 1; (iii) huấn luyện một bộ phân loại có-giám-sát chỉ trên dữ liệu Container khi kho tấn công đủ lớn, rồi so sánh với VAE (Container); (iv) ghép cảnh báo với hành động thực thi Sigkill của Tetragon để đo tỉ lệ ngăn chặn end-to-end trước khai thác CVE, không chỉ phát hiện.

IX. KẾT LUẬN

Chúng tôi đã tái hiện SyscallAD trên bộ dữ liệu công bố trong phạm vi nhiều lấy mẫu của các chỉ số tiêu đề, có điều kiện là thay một bộ chuẩn hoá không được nêu bằng một bộ scaler dải lượng phân vị nguyên tắc. Sau đó chúng tôi triển khai bộ phát hiện trên luồng sự kiện gRPC của Cilium Tetragon và quan sát thấy mô hình đã công bố không chuyển giao được sang các khối công việc container kiểu sản xuất: một vòng lặp **curl** lành tính bị gắn nhãn còn vòng lặp gắn trình gỡ lỗi thì không. Huấn luyện lại cùng kiến trúc VAE trên vết syscall thu từ khối công việc container thật loại bỏ sạch lỗi này. Đóng góp của thành phần iForest đối với khả năng phân biệt được chứng minh là nhỏ trong cả hai cấu hình, và các bộ phân loại có-giám-sát trên cùng không gian đặc trưng cạnh tranh được. Khuyến nghị thực tiễn là mọi triển khai bộ phát hiện kiểu SyscallAD nên đi trước bởi việc thu thập dữ liệu trong-môi-trường, và bộ chuẩn hoá cùng ngưỡng quyết định nên được khảo sát trên phân phối điểm của môi-trường-triển-khai thay vì kế thừa từ mặc định của môi-trường-huấn-luyện.

TÀI LIỆU

- [1] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, “DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time,” *IEEE Access*, vol. 13, 2025.
- [2] Cilium Authors, “Tetragon: eBPF-based security observability and runtime enforcement,” <https://github.com/cilium/tetragon>, 2025, version 1.7.0.
- [3] “DongTing: A linux kernel syscall anomaly detection dataset,” Zenodo, 2023, linux kernel test-suite traces (LTP, Kselftests, OpenPOSIX, glibc) paired with syzkaller PoC traces across 26 kernel releases.
- [4] Google, “syzkaller: An unsupervised coverage-guided kernel fuzzer,” <https://github.com/google/syzkaller>, 2024.
- [5] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, “A sense of self for Unix processes,” in *Proc. IEEE Symposium on Security and Privacy*, 1996, pp. 120–128.
- [6] A. Aldribi, I. Traoré, P. G. Quinan, and A. Sayed, “NIMOS: A network-based anomaly detection system using LSTM-based models for container workloads,” in *Proc. IEEE Conference on Communications and Network Security*, 2022, cited per DeSFAM [1].
- [7] S. Ghavamnia, T. Palit, S. Mishra, and M. Polychronakis, “Confine: Automated system call policy generation for container attack surface reduction,” in *Proc. International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, optimus baseline as referenced by DeSFAM [1].